

Universidad CENFOTEC

Escuela de Ingeniería del Software

Tarea 2: Cajero automático

SOFT-01 - Principios de programación 1

Julian Salazar Solís

Profesora: Veronica Mora Lezcano

Sección: SCV-6 – Periodo: 2026-C3

25 de Octubre, 2026

Enlace del repositorio:
https://github.com/julian26-salazar04/SOFT-01-Principios-de-Programaci-n-1.git

Rama de la solución: main

1. Problema seleccionado

Problema 1. Cajero automático

Un cajero automático permite al usuario realizar varios retiros consecutivos sobre un mismo saldo inicial hasta que decida salir o hasta que no tenga fondos suficientes. El saldo inicial de la cuenta es de ₡250 000.

El programa debe solicitar: monto a retirar en cada operación y confirmación para continuar retirando (s / n).

Reglas:

- El retiro mínimo por operación es de ₡5 000.
- El retiro máximo por operación es de ₡100 000.
- Si el monto supera el saldo disponible, no se realiza el retiro y se muestra un mensaje.
- El ciclo continúa mientras el usuario escriba “s” y exista saldo disponible.
- Al finalizar, se muestra el saldo restante y la cantidad de retiros exitosos.

El programa debe mostrar: por cada intento, si el retiro se realizó o no, el saldo actualizado, y al final el saldo restante y el total de retiros exitosos.

2. Análisis del problema

2.1 Tabla General

ENTRADA	PROCESO	SALIDA
Monto a retirar Respuesta s/n	Validar el monto (mínimo, máximo, saldo) Restar del saldo y contar retiros exitosos Repetir mientras respuesta = “s” y saldo > 0	Resultado de cada intento Saldo actualizado Saldo restante final Total de retiros exitosos

2.2 Constantes

CONSTANTE	VALOR	DESCRIPCIÓN
SALDO_INICIAL	250000	Saldo con el que inicia la cuenta (₡)
RETIRO_MINIMO	5000	Monto mínimo por operación (₡)
RETIRO_MAXIMO	100000	Monto máximo por operación (₡)

2.3 Tabla de datos de entrada

DATO	VARIABLE	TIPO	DESCRIPCIÓN
Respuesta inicial	respuesta	str	“s” para empezar a retirar, “n” para salir sin retirar
Monto a retirar	monto_retiro	int	Se pide en cada operación. Debe estar entre RETIRO_MINIMO y RETIRO MAXIMO y no superar el saldo
Confirmación para continuar	respuesta	str	Se pide después de cada intento: “s” continúa, otro valor termina

2.4 Tabla de variables de proceso

VARIABLE	TIPO	VALOR INICIAL	FUNCION
saldo_actual	int	SALDO_INICIAL	Saldo disponible, baja con cada retiro exitoso
retiros_exitosos	int	0	Contador de retiros realizados
respuesta	str	lo que escriba el usuario	Controla si el ciclo continúa

2.5 Reglas de decisión

#	CONDICIÓN	RESULTADO
1	monto_retiro < RETIRO_MINIMO	No se retira, mensaje de monto menor al mínimo
2	monto_retiro > RETIRO_MAXIMO	No se retira, mensaje de monto mayor al máximo
3	monto_retiro > saldo_actual	No se retira, mensaje de fondos insuficientes
4	Pasa las tres validaciones	saldo_actual = saldo_actual - monto_retiro y retiros_exitosos = retiros_exitosos + 1

2.6 Tabla de datos de salida

SALIDA	CUANDO SE MUESTRA	CONTENIDO
Resultado del intento	Dentro del ciclo, en cada intento	Si el retiro se realizó o no, y el motivo si no se realizó
Saldo actualizado	Dentro del ciclo, en cada intento	Valor de saldo_actual
Saldo restante	Al finalizar	Valor final de saldo_actual
Retiros exitosos	Al finalizar	Valor de retiros_exitosos

3. Algoritmo y diagrama de flujo

3.1 Pseudocódigo

```
INICIO

  CONSTANTES: SALDO_INICIAL = 250000

              RETIRO_MINIMO = 5000

              RETIRO_MAXIMO = 100000


  saldo_actual ← SALDO_INICIAL
```

```
retiros_exitosos ← 0

LEER respuesta   ("¿Desea realizar un retiro? (s/n)")

MIENTRAS respuesta = "s" Y saldo_actual > 0 HACER
   LEER monto_retiro

   SI monto_retiro < RETIRO_MINIMO ENTONCES
      MOSTRAR "Monto menor al mínimo. Retiro NO realizado"
   SINO SI monto_retiro > RETIRO_MAXIMO ENTONCES
      MOSTRAR "Monto mayor al máximo. Retiro NO realizado"
   SINO SI monto_retiro > saldo_actual ENTONCES
      MOSTRAR "Fondos insuficientes. Retiro NO realizado"
   SINO
      saldo_actual ← saldo_actual - monto_retiro
      retiros_exitosos ← retiros_exitosos + 1
      MOSTRAR "Retiro realizado"
   FIN SI

   MOSTRAR saldo_actual

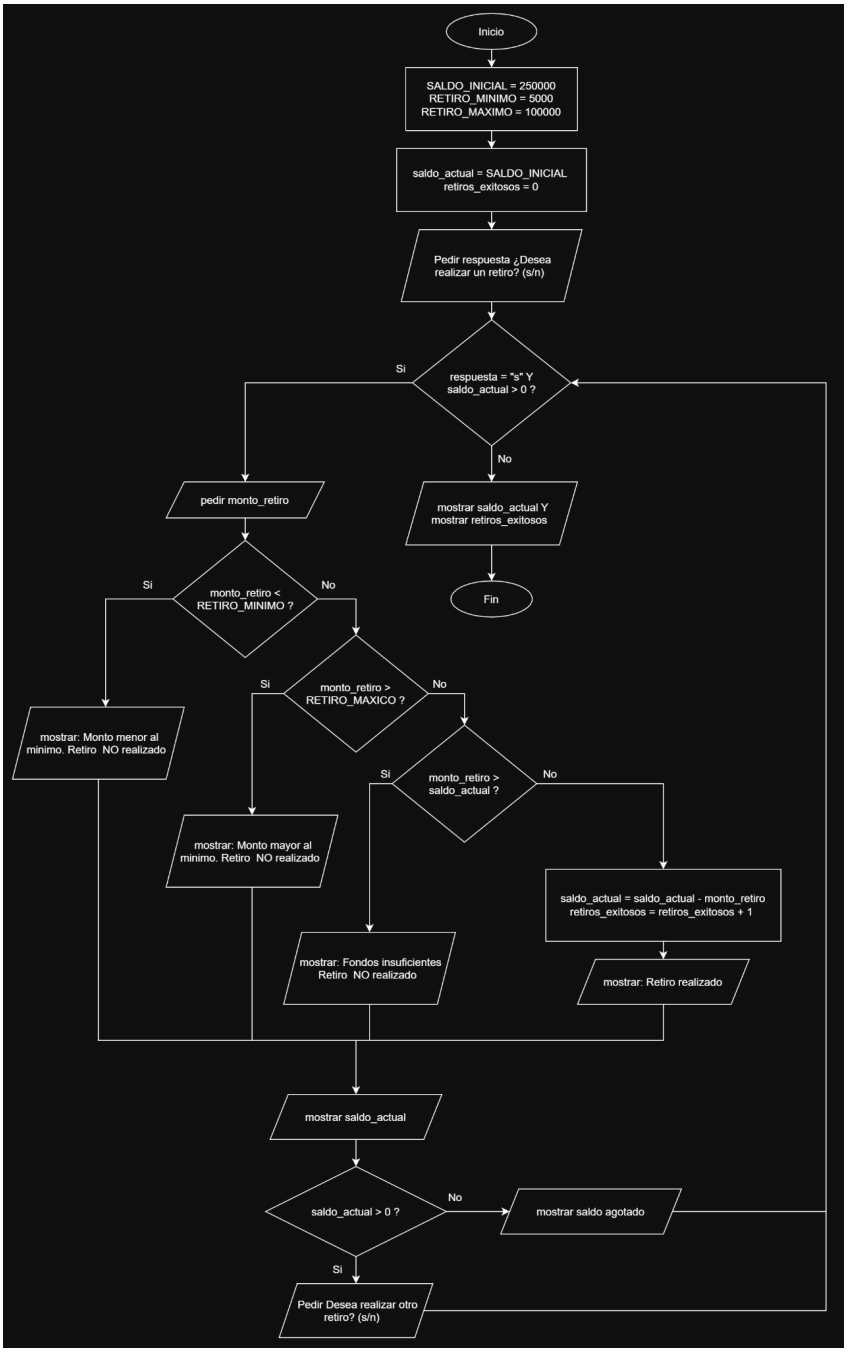
   SI saldo_actual > 0 ENTONCES
      LEER respuesta   ("¿Desea realizar otro retiro? (s/n)")
   SINO
      MOSTRAR "Saldo agotado"
   FIN SI
FIN MIENTRAS
```

MOSTRAR saldo_actual

MOSTRAR retiros_exitosos

FIN

3.2 Diagrama de flujo



4. Pruebas de escritorios

4.1 Traza A: Caso normal

ITERACIÓN	VARIABLES DE CONTROL	CONDICIÓN EVALUADA	¿ENTRA AL CICLO?	CÁLCULOS	ACUMULADORES	SALIDA MOSTRADA
0 (inicial)	respuesta = — saldo_actual = —	—	—	saldo_actual = 250000 retiros_exitosos = 0 Se lee respuesta = “s”	saldo_actual = 250000 retiros_exitosos = 0	¿Desea realizar un retiro? (s/n)
1	respuesta = “s” saldo_actual = 250000	“s” == “s” Y 250000 > 0: Verdadero	Si	Se lee monto_retiro = 100000 100000 < 5000: Falso 100000 > 100000: Falso 100000 > 250000: Falso saldo_actual = 250000 - 100000 = 150000 retiros_exitosos = 0 + 1 = 1 150000 > 0: Verdadero Se lee respuesta = “s”	saldo_actual = 150000 retiros_exitosos = 1	Retiro realizado Saldo: 150000
2	respuesta = “s” saldo_actual = 150000	“s” == “s” Y 150000 > 0: Verdadero	Si	Se lee monto_retiro = 100000 100000 < 5000: Falso 100000 > 100000: Falso 100000 > 150000: Falso saldo_actual = 150000 - 100000 = 50000 retiros_exitosos = 1 + 1 = 2 50000 > 0: Verdadero Se lee respuesta = “s”	saldo_actual = 50000 retiros_exitosos = 2	Retiro realizado Saldo: 50000
3	respuesta = “s” saldo_actual = 50000	“s” == “s” Y 50000 > 0: Verdadero	Si	Se lee monto_retiro = 80000 80000 < 5000: Falso 80000 > 100000: Falso 80000 > 50000: Verdadero No se modifica el saldo 50000 > 0: Verdadero Se lee respuesta = “s”	saldo_actual = 50000 retiros_exitosos = 2	Fondos insuficientes. Retiro NO realizado Saldo: 50000
4	respuesta = “s” saldo_actual = 50000	“s” == “s” Y 50000 > 0: Verdadero	Si	Se lee monto_retiro = 3000 3000 < 5000: Verdadero No se modifica el saldo 50000 > 0: Verdadero Se lee respuesta = “s”	saldo_actual = 50000 retiros_exitosos = 2	Monto menor al mínimo. Retiro NO realizado Saldo: 50000
5	respuesta = “s” saldo_actual = 50000	“s” == “s” Y 50000 > 0: Verdadero	Si	Se lee monto_retiro = 150000 150000 < 5000: Falso 150000 > 100000: Verdadero No se modifica el saldo 50000 > 0: Verdadero Se lee respuesta = “n”	saldo_actual = 50000 retiros_exitosos = 2	Monto mayor al máximo. Retiro NO realizado Saldo: 50000
Final	respuesta = “n” saldo_actual = 50000	“n” == “s” Y 50000 > 0: Falso	No: sale del ciclo	No se ejecuta el cuerpo del ciclo	saldo_actual = 50000 retiros_exitosos = 2	Saldo restante: 50000 Retiros exitosos: 2

4.2 Traza B: Cero iteraciones

ITERACIÓN	VARIABLES DE CONTROL	CONDICIÓN EVALUADA	¿ENTRA AL CICLO?	CÁLCULOS	ACUMULADORES	SALIDA MOSTRADA
0 (inicial)	respuesta = — saldo_actual = —	—	—	saldo_actual = 250000 retiros_exitosos = 0 Se lee respuesta = “n”	saldo_actual = 250000 retiros_exitosos = 0	¿Desea realizar un retiro? (s/n)
Final	respuesta = “n” saldo_actual = 250000	“n” == “s” Y 250000 > 0: Falso	No: sale del ciclo	No se ejecuta el cuerpo del ciclo	saldo_actual = 250000 retiros_exitosos = 0	Saldo restante: 250000 Retiros exitosos: 0

5. Código en python

```
# Nombre del estudiante: Julián Salazar Solis

# Contexto seleccionado: Problema 1. Cajero automático

# Descripción: permite realizar varios retiros consecutivos sobre un saldo

# inicial, valida cada monto y muestra el saldo restante y la cantidad de

# retiros exitosos al finalizar.



# --- Constantes ---

SALDO_INICIAL = 250000

RETIRO_MINIMO = 5000

RETIRO_MAXIMO = 100000



# --- Inicialización ---

saldo_actual = SALDO_INICIAL

retiros_exitosos = 0



respuesta = input("¿Desea realizar un retiro? (s/n): ")
```

```
# --- Ciclo principal ---
while respuesta == "s" and saldo_actual > 0:
    monto_retiro = int(input("Ingrese el monto a retirar (₡): "))

    if monto_retiro < RETIRO_MINIMO:
        print("Monto menor al mínimo. Retiro NO realizado")
    elif monto_retiro > RETIRO_MAXIMO:
        print("Monto mayor al máximo. Retiro NO realizado")
    elif monto_retiro > saldo_actual:
        print("Fondos insuficientes. Retiro NO realizado")
    else:
        saldo_actual = saldo_actual - monto_retiro
        retiros_exitosos = retiros_exitosos + 1
        print("Retiro realizado")

    print("Saldo:", saldo_actual)

    if saldo_actual > 0:
        respuesta = input("¿Desea realizar otro retiro? (s/n): ")
    else:
        print("Saldo agotado")

# --- Salida final ---
print("\n----------- Resumen -----------")
```

```
print("Saldo restante:", saldo_actual)

print("Retiros exitosos:", retiros_exitosos)
```

6. Pruebas del programa

#	CASO	QUE DEMUESTRA	ENTRADAS	SALDO FINAL	RETIROS EXITOSOS
1	Normal	El ciclo se ejecuta varias veces y pasa por las 4 ramas	s, 100000, s, 100000, s, 80000, s, 3000, s, 150000, n	50000	2
2	Cero iteraciones	El ciclo no se ejecuta ni una vez	n	250000	0
3	Una iteración	El ciclo se ejecuta exactamente una vez	s, 50000, n	200000	1
4	Límite	El saldo llega a 0 y el ciclo termina por saldo agotado	s, 100000, s, 100000, s, 50000	0	3

Prueba 1: Caso normal

Objetivo: comprobar el funcionamiento general, con retiros exitosos y las tres validaciones fallidas.

Datos de entrada: s, 100000, s, 100000, s, 80000, s, 3000, s, 150000, n

Resultado esperado:

¿Desea realizar un retiro? (s/n): s

Ingrese el monto a retirar (₡): 100000

Retiro realizado

Saldo: 150000

¿Desea realizar otro retiro? (s/n): s

Ingrese el monto a retirar (₡): 100000

Retiro realizado

Saldo: 50000

¿Desea realizar otro retiro? (s/n): s

Ingrese el monto a retirar (₡): 80000

Fondos insuficientes. Retiro NO realizado

Saldo: 50000

¿Desea realizar otro retiro? (s/n): s

Ingrese el monto a retirar (₡): 3000

Monto menor al mínimo. Retiro NO realizado

Saldo: 50000

¿Desea realizar otro retiro? (s/n): s

Ingrese el monto a retirar (₡): 150000

Monto mayor al máximo. Retiro NO realizado

Saldo: 50000

¿Desea realizar otro retiro? (s/n): n

---------- Resumen ----------

Saldo restante: 50000

Retiros exitosos: 2

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\ejercicios> &
 C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/x
mlsc/OneDrive/Desktop/Principios de programacion #1/ejercicios/ejercicios/tare
a2-cajero_automatico/cajero_automatico.py"
¿Desea realizar un retiro? (s/n): s
Ingrese el monto a retirar (₡): 100000
Retiro realizado
Saldo: 150000
¿Desea realizar otro retiro? (s/n): s
Ingrese el monto a retirar (₡): 100000
Retiro realizado
Saldo: 50000
¿Desea realizar otro retiro? (s/n): s
Ingrese el monto a retirar (₡): 80000
Fondos insuficientes. Retiro NO realizado
Saldo: 50000
¿Desea realizar otro retiro? (s/n): s
Ingrese el monto a retirar (₡): 3000
Monto menor al mínimo. Retiro NO realizado
Saldo: 50000
¿Desea realizar otro retiro? (s/n): s
Ingrese el monto a retirar (₡): 150000
Monto mayor al máximo. Retiro NO realizado
Saldo: 50000
¿Desea realizar otro retiro? (s/n): n

---------- Resumen ----------
Saldo restante: 50000
Retiros exitosos: 2
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\ejercicios>
```

Prueba 2: Cero iteraciones

Objetivo: comprobar que si el usuario no quiere retirar, el ciclo no se ejecuta y el saldo queda intacto.

Datos de entrada: n

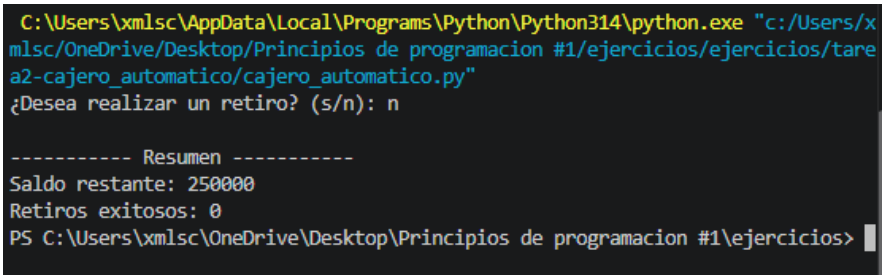
Resultado esperado:

¿Desea realizar un retiro? (s/n): n

---------- Resumen ----------

Saldo restante: 250000

Retiros exitosos: 0



Prueba 3: Una sola iteración

Objetivo: comprobar que el ciclo se ejecuta exactamente una vez.

Datos de entrada: s, 50000, n

Resultado esperado:

¿Desea realizar un retiro? (s/n): s

Ingrese el monto a retirar (₡): 50000

Retiro realizado

Saldo: 200000

¿Desea realizar otro retiro? (s/n): n

---------- Resumen ----------

Saldo restante: 200000

Retiros exitosos: 1

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\ejercicios> &
 C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/x
mlsc/OneDrive/Desktop/Principios de programacion #1/ejercicios/ejercicios/tare
a2-cajero_automatico/cajero_automatico.py"
¿Desea realizar un retiro? (s/n): s
Ingrese el monto a retirar (₡): 50000
Retiro realizado
Saldo: 200000
¿Desea realizar otro retiro? (s/n): n

---------- Resumen ----------
Saldo restante: 200000
Retiros exitosos: 1
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\ejercicios>
```

Prueba 4: Caso límite

Objetivo: comprobar que el ciclo termina cuando el saldo llega a 0, aunque el usuario no haya escrito “n”. Este caso también usa el retiro máximo permitido (₡100 000) y retira el saldo exacto.

Datos de entrada: s, 100000, s, 100000, s, 50000

Resultado esperado:

¿Desea realizar un retiro? (s/n): s

Ingrese el monto a retirar (₡): 100000

Retiro realizado

Saldo: 150000

¿Desea realizar otro retiro? (s/n): s

Ingrese el monto a retirar (₡): 100000

Retiro realizado

Saldo: 50000

¿Desea realizar otro retiro? (s/n): s

Ingrese el monto a retirar (₡): 50000

Retiro realizado

Saldo: 0

Saldo agotado

---------- Resumen ----------

Saldo restante: 0

Retiros exitosos: 3

```
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\ejercicios> &
 C:\Users\xmlsc\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/x
mlsc/OneDrive/Desktop/Principios de programacion #1/ejercicios/ejercicios/tare
a2-cajero_automatico/cajero_automatico.py"
¿Desea realizar un retiro? (s/n): s
Ingrese el monto a retirar (₡): 100000
Retiro realizado
Saldo: 150000
¿Desea realizar otro retiro? (s/n): s
Ingrese el monto a retirar (₡): 100000
Retiro realizado
Saldo: 50000
¿Desea realizar otro retiro? (s/n): s
Ingrese el monto a retirar (₡): 50000
Retiro realizado
Saldo: 0
Saldo agotado

---------- Resumen ----------
Saldo restante: 0
Retiros exitosos: 3
PS C:\Users\xmlsc\OneDrive\Desktop\Principios de programacion #1\ejercicios>
```